NAME

grouper.py − find groups of directories with identical or similar contents in a sumtag database

SYNOPSIS

grouper.py −−database *DB* [*ACTION*] [*OPTIONS*]

DESCRIPTION

grouper.py is an experimental companion program to **sumtag**(1). It reads the **files/mountpoints** tables of a sumtag SQLite database and finds groups of directories whose contents are identical or nearly so — for example, two slightly different copies of the same project, or the five scattered backups of a family video archive. It never touches files, extended attributes, or anything else on disk: everything it knows comes from the database, and everything it builds (its derived tables) lives inside the same database file.

−−database is required on every invocation. With no action flags, the stored grouping is printed (see THE REPORT). Building that grouping is a three-stage pipeline, each stage persisted so later stages are cheap to re-run:

−−index
: Stage 1: distill the file table into the distinct directories that directly contain at least one visible stamped file. Direct children only, deliberately: a directory's own file listing is its signature. Two junk filters apply, each reporting its dropped count on standard error: a directory whose path contains a hidden (dot-prefixed) component — **.git/**hooks, **.build/**checkouts, anything under **.Trash−*** — is never indexed at all, and a directory whose direct files are all hidden is dropped. **−−no−junk−filter** disables both.

−−pairs
: Stage 2: compare every directory to every other with the selected comparison function and store every nonzero similarity. This N²/2 comparison is the expensive part, so the similarity threshold is deliberately not applied here — regrouping at a different threshold recomputes nothing.

−−threshold *X*
: Stage 3: walk the stored pairs best-first and partition directories into groups, taking *X* (0.0−1.0) as the similar-enough bar. Each directory gets exactly one placement decision and groups never merge, so the result is a true partition. The grouping is persisted, and the run ends with the report. If the stored grouping is already current for this threshold, the rebuild is skipped.

−−prep runs stages 1 and 2 together, and stages combine freely in one invocation (**−−prep −−threshold** 0.7). An interrupted **−−pairs** keeps its completed work, but its provenance row is written only on success, so a partial pair table is refused by **−−threshold** rather than silently grouped from.

OPTIONS

−−database *DB*
: Path to the sumtag SQLite database. Required.

−−prep
: Stages 1+2: **−−index** then **−−pairs**.

−−index
: Stage 1 alone: (re)build the directory index. Silent on success.

−−no−junk−filter
: Disable **−−index**'s junk filtering for the run: directories under hidden path components and all-hidden directories are indexed like any other. The escape hatch for deliberately grouping hidden trees — for example, asking whether trash contents are duplicated elsewhere before emptying the trash. The filtering regime the index was built under is recorded in the database.

−−pairs
: Stage 2 alone: score and store directory pairs. Silent on success.

−−threshold *X*
: Stage 3: build and persist the grouping at similarity bar *X*, then report it. Refused if *X* is below the pair table's stored **−−min−sim** floor (the stored pairs could not answer honestly).

−−fn *NAME*
: Comparison function for **−−pairs/−−compare**. One of: **digest** (content only — renames are free), **name−digest** (all-or-nothing per name-and-digest pair), or **name−score** (the default; name-anchored partial credit: one point for a shared basename, two more if the digests also agree).

−−jobs *N*
: Worker processes for **−−pairs** scoring (default: all CPUs); 1 disables multiprocessing. Each worker loads its own full copy of the signature table, so on a big corpus this is a memory knob as much as a speed knob. The stored pair set is identical at any value.

−−progress
: Live progress bar on standard error for the build stages, following the **sumtag**(1) conventions: appears once a stage has run two seconds, redrawn in place, cleared on completion, suppressed when standard error is not a terminal. Shows work items done/total with rate, percentage, elapsed time, and ETA.

−−max−df *N*
: For **−−pairs**: instead of exhaustive all-pairs, score only candidate pairs sharing at least one signature token (basename or digest, per the comparison function) present in at most *N* directories. Nominated pairs get exact scores — the cap governs who gets looked at, never what a look sees; what is lost is only pairs whose sole overlap is ubiquitous tokens, whose similarity is necessarily tiny. By default, exhaustive comparison is used up to roughly ten thousand directories, then candidate nomination with *N*=1000; 0 forces exhaustive at any size. Candidate mode is announced on standard error and recorded in the database.

−−min−sim *X*
: For **−−pairs**: store only pairs scoring at least *X* (0.0−1.0). By default every nonzero pair is stored, which keeps **−−threshold** a free exploratory knob — but on an archive-scale corpus the weak tail dwarfs everything interesting. The floor governs what is kept, never what is looked at; it is recorded in the database, and a later **−−threshold** below it is refused with a rerun hint.

−−sort *KEY*
: Order for the report's groups. One of: **bond** (default) — creation order, strongest bonds first; **files** — each group's total stamped-file count, largest first; **size** — each group's total byte size, largest first; **tree−size** — like **size** but over each member directory's entire subtree. See THE REPORT.

−−ls *DIR*
: List the files the index records for one directory (absolute or mount-relative path).

−−compare *DIR_A DIR_B*
: Print the similarity (0.0−1.0) of two directories' contents under the selected comparison function.

−−dups
: Report duplicate files (same digest) across the whole database — files, not directories.

−−top [*N*]
: Report the *N* (default 1) most frequently occurring checksums and their files, excluding empty files.

−−min *N*
: For **−−dups/−−top**: only report groups of at least *N* files (default 2).

THE REPORT

The report opens with the grouping's provenance (comparison function, threshold, build time), then prints each group with its member directories:

```
group 17  (2 directories, 1852 files, 908.4GiB in tree, best pair 1.000)
    /mnt/backup/recovered/media
    /mnt/backup/master/media
```

best pair is the highest stored similarity between any two members — the bond that says how alike the group is at its closest point. Members are listed alphabetically by path.

−−sort is display-time only: nothing is stored or rebuilt, and group numbering does not change. The default **bond** order lists groups in creation order, strongest bonds first. The **files**, **size**, and **tree−size** orders list the largest groups first and add each group's file count and byte total to its header line. **size** counts the member directories' direct children — the same files the similarity was scored over — while **tree−size** rolls up each member's entire subtree (marked **in tree** in the header): members nested under another member of the same group are skipped so nothing is double-counted, and the totals include stamped files the index itself never looked at (hidden files, dropped junk directories). Under **tree−size**, the same bytes can legitimately appear in several groups' totals, so report-wide sums are not meaningful.

Byte totals read the **size** column that a **sumtag −−locate** run populates. If some files lack sizes, the report proceeds with a note on standard error that totals undercount; if all do, a size sort is refused outright rather than producing a silently meaningless ordering.

EXAMPLES

Build everything and group at a moderate similarity bar, in one invocation:

```
grouper.py --database /var/lib/sumtag.sqlite --prep --threshold 0.7
```

Tighten the grouping to near-identical directories only. The expensive pair table is already stored, so this regroups in seconds and recomputes nothing:

```
grouper.py --database /var/lib/sumtag.sqlite --threshold 0.9999
```

Print the stored grouping again, touching nothing:

```
grouper.py --database /var/lib/sumtag.sqlite
```

The space-hunting view: the same grouping ordered by each group's total subtree size, so the report leads with the largest duplicated trees — typically worth capturing to a file:

```
grouper.py --database /var/lib/sumtag.sqlite --sort tree-size > groups.out
```

Rebuild the pair table on an archive-scale corpus: candidate nomination (share a token seen in at most 1000 directories), keep only pairs scoring at least 0.5, cap worker memory at four processes, and watch progress:

```
grouper.py --database big.sqlite --pairs --max-df 1000 \
    --min-sim 0.5 --jobs 4 --progress
```

Ask why (or how much) two specific directories resemble each other, and see what the index recorded for one of them:

```
grouper.py --database db.sqlite --compare /mnt/a/proj /mnt/b/proj
grouper.py --database db.sqlite --ls /mnt/a/proj
```

Duplicate files rather than directories — every digest shared by at least three files, then the ten most frequent checksums:

```
grouper.py --database db.sqlite --dups --min 3
grouper.py --database db.sqlite --top 10
```

Content-only grouping, so wholesale renames still match:

```
grouper.py --database db.sqlite --pairs --fn digest
grouper.py --database db.sqlite --threshold 0.9
```

Rebuild everything with junk filtering off, to ask whether hidden trees (here, trash contents) are duplicated elsewhere:

```
grouper.py --database db.sqlite --no-junk-filter --prep --threshold 0.9
```

EXIT STATUS

0 on success; **1** if an error prevented the request from being answered (not a sumtag database, no stored pairs or grouping yet, a **−−threshold** below the stored **−−min−sim** floor, or a size sort with no stored sizes).

CAVEATS

The derived tables are snapshots: when sumtag rescans and the file table changes, rerun the pipeline (**−−prep**, then **−−threshold**). The report prints a note when the pair table is newer than the stored

grouping, and when the directory index is newer than the pair table (for example, after reindexing under a different junk-filtering regime). The index references file rows by SQLite rowid, which **VACUUM** can renumber — acceptable for artifacts rebuilt in one command.

The database must have been populated by **sumtag**(1) first (e.g. **sumtag −−database db.sqlite −−sum −−locate /data**); byte totals in the report additionally want the stat metadata a **−−locate** pass records.

SEE ALSO

sumtag(1)

AUTHOR

The sumtag authors.